

1. a. 實現高階語言中的 if-else 邏輯

b.

JZ	Jump if zero
JE	Jump if equal
JA	Jump if above
JG	Jump if greater
JC	Jump if carry

c. JZ: Jump if zero ($ZF=1$)

JNZ: Jump if not zero ($ZF=0$)

2.

a. LOOPE destination

→ { 1. $ECX \leftarrow ECX - 1$
2. if $ECX > 0$ and $ZF=1$, jump to destination

b. while loop: 先檢查條件符合後, 才執行 body

do-while loop: 先執行 body, 再判斷條件

c. `mov ecx, 5 ; for(int i=0; i<5; i++)`
`mov eax, 0`

L1: `call WriteDec`
`call Crlf`
`inc eax`
`loop L1`

3.

a. CMP 影響 ZF, CF

TEST 只影響 ZF

b. CMP: subtraction of source from destination and destination operand is not changed

① destination == source

⇒ ZF=1, CF=0

② destination < source

⇒ ZF=0, CF=1

③ destination > source

⇒ ZF=0, CF=0

TEST: 兩個運算元做 AND 後

判斷是否為 0

如果為 0, ZF=1

如果不為 0, ZF=0

c.

mov al, 6

cmp al, 5 ; 6-5=1, ZF=0, CF=0

4.

a. AND

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

XOR

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

NOT

x	$\neg x$
0	1
1	0

b. 透過 AND, OR, XOR 改變 EFLAGS

c. `mov al, 10101010b`

`AND al, 00001111b`

```

10101010
AND 00001111
-----
00000101

```

Practical Exercises

1.

```
1      include Irvine32.inc
2
3      .data
4          promptMsg    BYTE "Enter an integer: ",0
5          posMsg       BYTE "The number is positive.",0
6          negMsg       BYTE "The number is negative.",0
7          zeroMsg      BYTE "The number is zero.",0
8
9      .code
10     main PROC
11
12         ; ask for input
13         mov edx, OFFSET promptMsg
14         call WriteString
15
16         ; read input
17         call ReadInt
18
19         ; =0
20         cmp eax, 0
21         je  IsZero
22
23         ; >0
24         jg  IsPositive
25
26         ; <0
27         jl  IsNegative
28
```

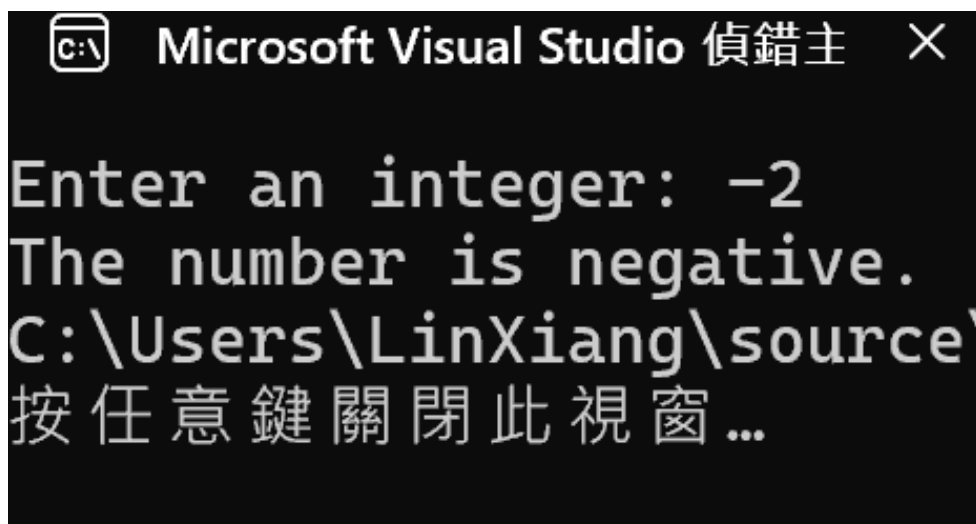
```
28
29     IsPositive:
30         mov edx, OFFSET posMsg
31         call WriteString
32         jmp EndProgram
33
34     IsNegative:
35         mov edx, OFFSET negMsg
36         call WriteString
37         jmp EndProgram
38
39     IsZero:
40         mov edx, OFFSET zeroMsg
41         call WriteString
42
43     EndProgram:
44         exit
45     main ENDP
46
47     END main
```

Output:



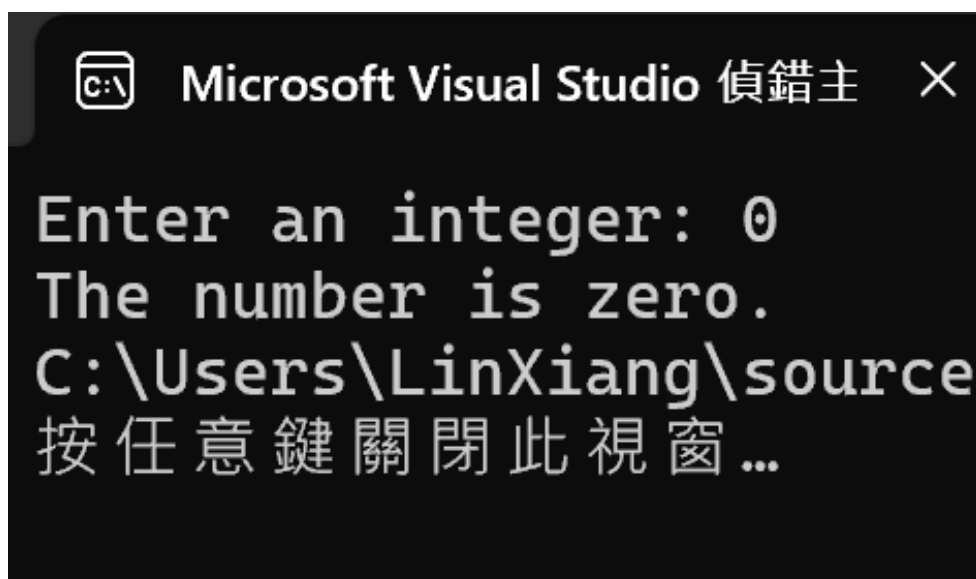
Microsoft Visual Studio 偵錯主 ×

Enter an integer: 5
The number is positive.
C:\Users\LinXiang\source\
按任意鍵關閉此視窗...



Microsoft Visual Studio 偵錯主 ×

Enter an integer: -2
The number is negative.
C:\Users\LinXiang\source\
按任意鍵關閉此視窗...



Microsoft Visual Studio 偵錯主 ×

Enter an integer: 0
The number is zero.
C:\Users\LinXiang\source\
按任意鍵關閉此視窗...

2.(1) 解法一

```
1      include Irvine32.inc
2
3      .code
4      main PROC
5          mov ebx,10
6          mov eax,1
7          mov ecx,0
8
9      top:    cmp eax,ebx
10             ja EndProgram
11             add ecx,eax
12             inc eax
13             jmp top
14
15             EndProgram:
16                 mov eax,ecx
17                 call WriteDec
18                 exit
19      main ENDP
20      END main
```

Output



```
Microsoft Visual Studio 偵錯主 ×
55
C:\Users\LinXiang\source
按任意鍵關閉此視窗...
```

2.(2) 解法一

```
1      include Irvine32.inc
2
3      .code
4      main PROC
5          mov ebx,0
6          mov eax,10
7
8
9      top:    cmp eax,ebx
10             jbe EndProgram
11             call WriteDec
12             call crlf
13             dec eax
14             jmp top
15
16             EndProgram:
17                 exit
18     main ENDP
19     END main
```

Output



The screenshot shows the output window of Microsoft Visual Studio. The title bar reads "Microsoft Visual Studio 偵錯主" with standard window controls. The output text is as follows:

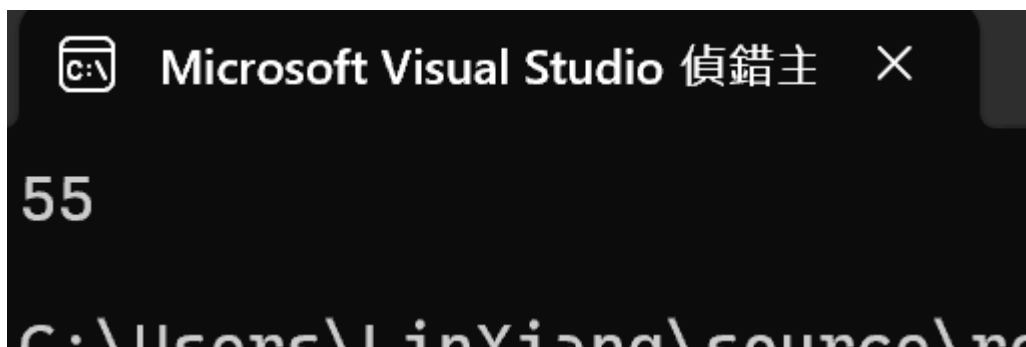
```
10
9
8
7
6
5
4
3
2
1

C:\Users\LinXiang\source\repo
按任意鍵關閉此視窗...
```

2.(1) 解法二

```
1      include Irvine32.inc
2
3      .data
4      sum DWORD 0
5
6      .code
7      main PROC
8          mov eax, 1
9          mov sum, 0
10
11         .while eax <= 10
12             add sum, eax
13             inc eax
14         .endw
15
16         mov eax, sum
17         call WriteDec
18         call Crlf
19
20         exit
21     main ENDP
22     END main
```

Output:



2.(2) 解法二

```
1      include Irvine32.inc
2
3      .code
4      main PROC
5          mov eax, 10
6
7          .repeat
8              call WriteDec
9              call Crlf
10             dec eax
11         .until eax == 0
12
13         exit
14     main ENDP
15 END main
16
```

Output:



The screenshot shows the output window of Microsoft Visual Studio. The title bar reads "Microsoft Visual Studio 偵錯主" with a close button. The output displays the numbers 10, 9, 8, 7, 6, 5, 4, 3, 2, and 1, each on a new line, representing the sequence of values printed by the program.

3.(1)

```
1      include Irvine32.inc
2
3      .data
4      msgEqual    BYTE "Two numbers are equal", 0
5      msgEaxBig   BYTE "EAX is greater than EBX", 0
6      msgEbxBig   BYTE "EBX is greater than EAX", 0
7
8      .code
9      main PROC
10         mov eax, 20
11         mov ebx, 10
12
13         cmp eax, ebx
14         je  Equal
15         ja  EaxGreater
16         jb  EbxGreater
17
18     Equal:
19         mov edx, OFFSET msgEqual
20         call WriteString
21         jmp EndProgram
22
23     EaxGreater:
24         mov edx, OFFSET msgEaxBig
25         call WriteString
26         jmp EndProgram
27
28     EbxGreater:
29         mov edx, OFFSET msgEbxBig
30         call WriteString
31
32     EndProgram:
33         exit
34     main ENDP
35     END main
36
```

Output:



```
Microsoft Visual Studio 偵錯主 ×
EAX is greater than EBX
C:\Users\LinXiang\source\r
按任意鍵關閉此視窗...
```

3.(2)

```
1      include Irvine32.inc
2
3      .data
4      msgEven BYTE "The number is EVEN", 0
5      msgOdd  BYTE "The number is ODD", 0
6
7      .code
8      main PROC
9          mov eax, 25
10
11         test eax, 1
12         jz   IsEven
13         jmp  IsOdd
14
15     IsEven:
16         mov edx, OFFSET msgEven
17         call WriteString
18         jmp EndProgram
19
20     IsOdd:
21         mov edx, OFFSET msgOdd
22         call WriteString
23
24     EndProgram:
25         invoke ExitProcess, 0
26     main ENDP
27     END main
```

Output:



4.(1)

```
1      include Irvine32.inc
2
3      .data
4      num          DWORD 500
5      msgIn        BYTE " is within the range (100-1000)", 0
6      msgOut       BYTE " is out of the range (100-1000)", 0
7
8      .code
9      main PROC
10         mov eax, num
11
12         cmp eax, 100
13         jl  OutOfRange
14
15         cmp eax, 1000
16         jg  OutOfRange
17
18         call WriteDec
19         mov edx, OFFSET msgIn
20         call WriteString
21         jmp EndProgram
22
23     OutOfRange:
24         mov edx, OFFSET msgOut
25         call WriteString
26
27     EndProgram:
28         call Crlf
29         exit
30     main ENDP
```

Output:



The screenshot shows the Microsoft Visual Studio interface. The title bar reads "Microsoft Visual Studio 偵錯主" with a close button. The output window displays the text "500 is within the range (100-1000)". Below this, the file path "C:\Users\LinXiang\source\repos\Asm_hw\Debug\A" is visible, followed by the instruction "按任意鍵關閉此視窗...".

4.(2)

```

1      Include Irvine32.inc
2
3      .data
4      menuMsg      BYTE "Menu:",0dh,0ah
5                   BYTE "1. Option One",0dh,0ah
6                   BYTE "2. Option Two",0dh,0ah
7                   BYTE "3. Option Three",0dh,0ah
8                   BYTE "Enter choice (1-3): ",0
9
10     msg1          BYTE "You chose Option One",0
11     msg2          BYTE "You chose Option Two",0
12     msg3          BYTE "You chose Option Three",0
13     msgInvalid    BYTE "Invalid choice",0
14
15     JumpTable     DWORD Option1, Option2, Option3
16

```

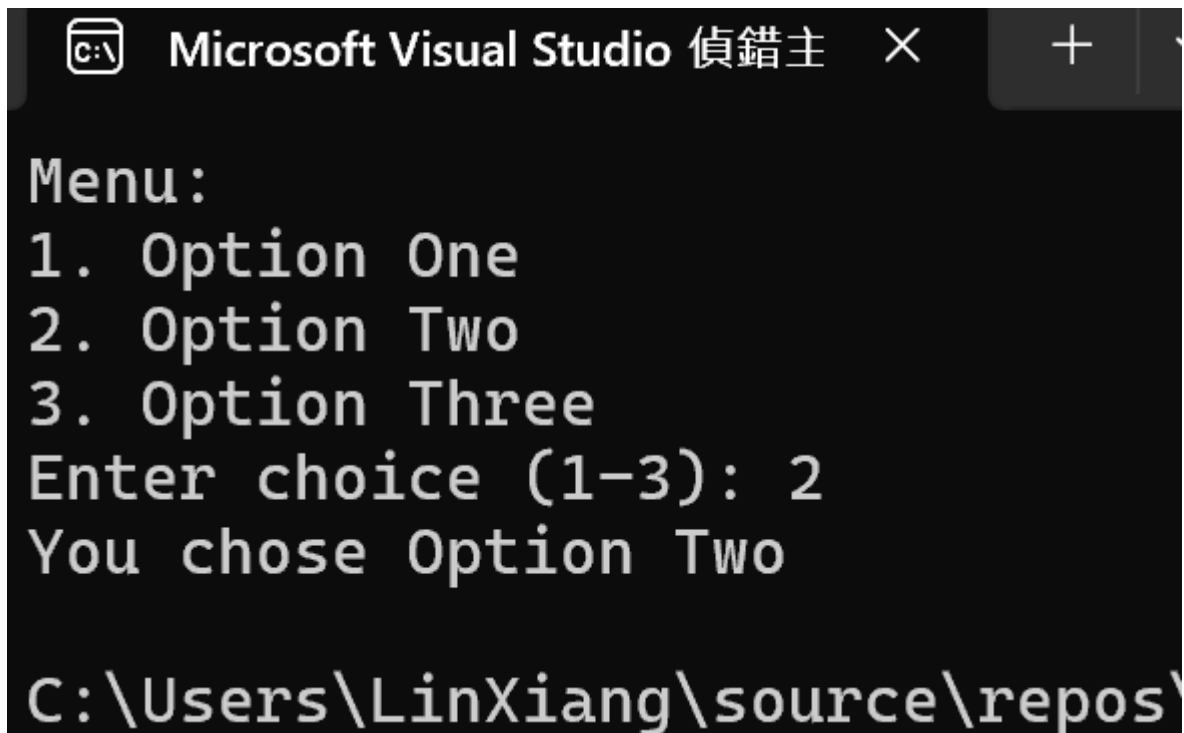
```

17     .code
18     main PROC
19         mov edx, OFFSET menuMsg
20         call WriteString
21
22         call ReadInt
23         dec eax          ; 1 - 3 → 0 - 2
24
25         cmp eax, 2
26         ja  InvalidChoice
27
28         mov ebx, OFFSET JumpTable
29         call DWORD PTR [ebx + eax*4] ; CALL procedures
30         jmp EndProgram
31
32     InvalidChoice:
33         mov edx, OFFSET msgInvalid
34         call WriteString
35
36     EndProgram:
37         call Crlf
38         exit
39     main ENDP

```

```
40
41 ; ===== Procedures =====
42 Option1 PROC
43     mov edx, OFFSET msg1
44     call WriteString
45     ret
46 Option1 ENDP
47
48 Option2 PROC
49     mov edx, OFFSET msg2
50     call WriteString
51     ret
52 Option2 ENDP
53
54 Option3 PROC
55     mov edx, OFFSET msg3
56     call WriteString
57     ret
58 Option3 ENDP
59
60 END main
61
```

Output:



The screenshot shows a terminal window titled "Microsoft Visual Studio 偵錯主" (Microsoft Visual Studio Debugger). The output text is as follows:

```
Menu:
1. Option One
2. Option Two
3. Option Three
Enter choice (1-3): 2
You chose Option Two

C:\Users\LinXiang\source\repos\
```

Assembly Language

Homework 6

Conceptual Questions

1. **Conditional Jumps:**
 - a. Explain the purpose of conditional jump instructions in assembly language.
 - b. List and describe at least five different conditional jump instructions.
 - c. How do the JZ and JNZ instructions work?
2. **Loops:**
 - a. Describe how loops are implemented in assembly language using conditional jumps.
 - b. What is the difference between a `while` loop and a `do-while` loop in assembly language?
 - c. Provide an example of a `for` loop implemented in assembly language.
3. **Comparing and Testing:**
 - a. Explain the difference between the `CMP` and `TEST` instructions.
 - b. How do the `CMP` and `TEST` instructions affect the flags in the EFLAGS register?
 - c. Provide an example of using the `CMP` instruction to compare two values.
4. **Boolean and Comparison Instructions:**
 - a. List and describe the purpose of the `AND`, `OR`, `XOR`, and `NOT` instructions.
 - b. How are these instructions used in conditional processing?
 - c. Provide an example of using the `AND` instruction to mask bits in a register.

Practical Exercises

1. **Conditional Jumps:** Write an assembly program that:
 - Reads an integer from the user.
 - Checks if the integer is positive, negative, or zero using conditional jumps.
 - Prints an appropriate message for each case.
2. **Loops:** Write an assembly program that:
 - Uses a `while` loop to calculate the sum of integers from 1 to 10.
 - Uses a `do-while` loop to print the numbers from 10 to 1.
3. **Comparing and Testing:** Write an assembly program that:

- Compares two integers and prints whether they are equal, or which one is greater.
- Uses the TEST instruction to check if a number is even or odd.

4. **High-Level Logic Structures:** Write an assembly program that:

- Implements an `if-else` statement to check if a number is within a specific range.
- Uses a jump table to implement a simple menu system with three options.